

Informe de Auditoría de Seguridad: WebGoat 8.1

Realizado por: Nicolás Navares

Fecha: 12 de enero de 2026

Proyecto: Auditoría web básica - Módulo Introducción a la Ciberseguridad

1. Ámbito y alcance de la auditoría

Este análisis se centra en la aplicación web **WebGoat 8.1.0**, desplegada mediante un contenedor **Docker** en un entorno local (127.0.0.1:8080). El objetivo es identificar y documentar las vulnerabilidades críticas indicadas en el plan de estudios, siguiendo el estándar del **OWASP Top 10**.

2. Informe ejecutivo

a. Breve resumen del proceso realizado

Se ha llevado a cabo un análisis de seguridad que comenzó con una fase de reconocimiento de red para identificar la infraestructura del servidor. Posteriormente, se procedió a la explotación manual de fallos de inyección, configuración y autenticación, utilizando herramientas como **Nmap** y **Burp Suite** para validar los riesgos detectados.

b. Vulnerabilidades destacadas

- **Inyecciones (A3):** Manipulación de consultas SQL para extraer información masiva y modificar datos financieros.
- **Configuración XML Incorrecta (A5):** Uso de entidades externas (XXE) para comprometer la privacidad del sistema de archivos.
- **Gestión de Autenticación (A7):** Mecanismos de recuperación de cuenta débiles basados en información predecible.

c. Conclusiones

La aplicación WebGoat presenta un nivel de riesgo elevado. La ausencia de mecanismos de validación y sanitización en las entradas del usuario permite que un atacante comprometa la integridad y confidencialidad de la base de datos y del sistema operativo.

d. Recomendaciones generales

Es muy importante implementar el principio de "defensa en profundidad", aplicando validaciones tanto en el lado del cliente como del servidor, utilizando consultas parametrizadas y reforzando los controles de acceso con factores de autenticación múltiple.

3. Descripción del proceso de auditoría

a. Reconocimiento / Information gathering

Se realizó un escaneo con **Nmap** para determinar los servicios activos .

- **Puertos:** Puerto **8080** abierto (Apache Tomcat) y puerto **9090** (Zeus-admin).
- **Sistema Operativo:** Se identificó un entorno **Linux 2.6.X|5.X**.

```
(n0v4r3k@N0v4r3k)~$ nmap -O 127.0.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-19 19:31 WET
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000019s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
8080/tcp   open  http-proxy
9090/tcp   open  zeus-admin
Device type: general purpose
Running: Linux 2.6.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:2.6.32 cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel:6
OS details: Linux 2.6.32, Linux 5.0 - 6.2
Network Distance: 0 hops

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.66 seconds

(n0v4r3k@N0v4r3k)~$
```

b. Explotación de vulnerabilidades detectadas

1. A3 Injection - SQL Injection (Confidencialidad)

- **Proceso:** Se inyectó la cadena ' OR 1=1 -- en el campo de búsqueda de empleados. El sistema, al no filtrar la entrada, aceptó la condición como verdadera para todos los registros.

Compromising confidentiality with String SQL injection

If a system is vulnerable to SQL injections, aspects of that system's CIA triad can be easily compromised (if you are unfamiliar with the CIA triad, check out the CIA triad lesson in the general category). In the following three lessons you will learn how to compromise each aspect of the CIA triad using techniques like *SQL string injections* or *query chaining*.

In this lesson we will look at **confidentiality**. Confidentiality can be easily compromised by an attacker using SQL injection; for example, successful SQL injection can allow the attacker to read sensitive data like credit card numbers from a database.

What is String SQL injection?

If an application builds SQL queries simply by concatenating user supplied strings to the query, the application is likely very susceptible to String SQL injection.

More specifically, if a user supplied string simply gets concatenated to a SQL query without any sanitization or preparation, then you may be able to modify the query's behavior by simply inserting quotation marks into an input field. For example, you could end the string parameter with quotation marks and input your own SQL after that.

It is your turn!

You are an employee named John **Smith** working for a big company. The company has an internal system that allows all employees to see their own internal data such as the department they work in and their salary.

The system requires the employees to use a unique *authentication TAN* to view their data.

Your current TAN is **3SL99A**.

Since you always have the urge to be the most highly paid employee, you want to exploit the system so that instead of viewing your own internal data, you want to take a look at the data of all your colleagues to check their current salaries.

Use the form below and try to retrieve all employee data from the **employees** table. You should not need to know any specific names or TANs to get the information you need.

You already found out that the query performing your request looks like this:

```
"SELECT * FROM employees WHERE last_name = '' + name + '' AND auth_tan = '' + auth_tan + ''";
```

Employee Name:
Authentication TAN:

Sorry the solution is not correct, please try again.

malformed string: '' AND auth_tan = ''

Compromising confidentiality with String SQL injection

If a system is vulnerable to SQL injections, aspects of that system's CIA triad can be easily compromised (if you are unfamiliar with the CIA triad, check out the CIA triad lesson in the general category). In the following three lessons you will learn how to compromise each aspect of the CIA triad using techniques like *SQL string injections* or *query chaining*.

In this lesson we will look at **confidentiality**. Confidentiality can be easily compromised by an attacker using SQL injection; for example, successful SQL injection can allow the attacker to read sensitive data like credit card numbers from a database.

What is String SQL injection?

If an application builds SQL queries simply by concatenating user supplied strings to the query, the application is likely very susceptible to String SQL injection.

More specifically, if a user supplied string simply gets concatenated to a SQL query without any sanitization or preparation, then you may be able to modify the query's behavior by simply inserting quotation marks into an input field. For example, you could end the string parameter with quotation marks and input your own SQL after that.

It is your turn!

You are an employee named John **Smith** working for a big company. The company has an internal system that allows all employees to see their own internal data such as the department they work in and their salary.

The system requires the employees to use a unique *authentication TAN* to view their data.

Your current TAN is **3SL99A**.

Since you always have the urge to be the most highly paid employee, you want to exploit the system so that instead of viewing your own internal data, you want to take a look at the data of all your colleagues to check their current salaries.

Use the form below and try to retrieve all employee data from the **employees** table. You should not need to know any specific names or TANs to get the information you need.

You already found out that the query performing your request looks like this:

```
"SELECT * FROM employees WHERE last_name = '' + name + '' AND auth_tan = '' + auth_tan + ''";
```

Employee Name:
Authentication TAN:

You have succeeded! You successfully compromised the confidentiality of data by viewing internal information that you should not have access to. Well done!

USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN
32147	Paulina	Travers	Accounting	46000	P45JSI
34477	Abraham	Holman	Development	50000	UU2ALK
37648	John	Smith	Marketing	64350	3SL99A
89762	Tobi	Barnett	Development	77000	TA9LL1
96134	Bob	Franco	Marketing	83700	LO9S2V

- **Conclusiones:** La falta de sanitización permite saltarse la lógica de la consulta, exponiendo datos privados como salarios y códigos TAN de toda la plantilla.
- **Recomendaciones / Mitigación:** Se debe utilizar **sentencias preparadas (Prepared Statements)** para que el motor de la base de datos trate la entrada del usuario como texto y no como código ejecutable.

2. A3 Injection - SQL Injection (Integridad - Query Chaining)

- **Proceso:** Se utilizó el comando ; UPDATE employees SET salary = 9000000 WHERE last_name = 'Smith' -- para encadenar una segunda instrucción a la consulta original.

Smith' ; UPDATE employees SET salary = 9000000 WHERE last_name = 'Smith' --

◂ 1 2 3 4 5 6 7 8 9 10 11 12 13 ▸

Compromising Integrity with Query chaining

After compromising the confidentiality of data in the previous lesson, this time we are gonna compromise the **integrity** of data by using **SQL query chaining**.

If a severe enough vulnerability exists, SQL injection may be used to compromise the integrity of any data in the database. Successful SQL injection may allow an attacker to change information that he should not even be able to access.

What is SQL query chaining?

Query chaining is exactly what it sounds like. With query chaining, you try to append one or more queries to the end of the actual query. You can do this by using the ; metacharacter. A ; marks the end of a SQL statement; it allows one to start another query right after the initial query without the need to even start a new line.

It is your turn!

You just found out that Tobí and Bob both seem to earn more money than you! Of course you cannot leave it at that. Better go and *change your own salary so you are earning the most!*

Remember: Your name is John **Smith** and your current TAN is **3SL99A**.

Employee Name:

Authentication TAN:

Get department

Still not earning enough or only your own salary must be changed! Better try again and change that.

USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN
96134	Bob	Franco	Marketing	83700	LO9S2V
89762	Tobi	Barnett	Development	77000	TA9LL1
37648	John	Smith	Marketing	64350	3SL99A
34477	Abraham	Holman	Development	50000	UU2ALK
32147	Paulina	Travers	Accounting	46000	P45JSI

◂ 1 2 3 4 5 6 7 8 9 10 11 12 13 ▸

Compromising Integrity with Query chaining

After compromising the confidentiality of data in the previous lesson, this time we are gonna compromise the **integrity** of data by using **SQL query chaining**.

If a severe enough vulnerability exists, SQL injection may be used to compromise the integrity of any data in the database. Successful SQL injection may allow an attacker to change information that he should not even be able to access.

What is SQL query chaining?

Query chaining is exactly what it sounds like. With query chaining, you try to append one or more queries to the end of the actual query. You can do this by using the ; metacharacter. A ; marks the end of a SQL statement; it allows one to start another query right after the initial query without the need to even start a new line.

It is your turn!

You just found out that Tobí and Bob both seem to earn more money than you! Of course you cannot leave it at that. Better go and *change your own salary so you are earning the most!*

Remember: Your name is John **Smith** and your current TAN is **3SL99A**.

✓ Employee Name:

Authentication TAN:

Get department

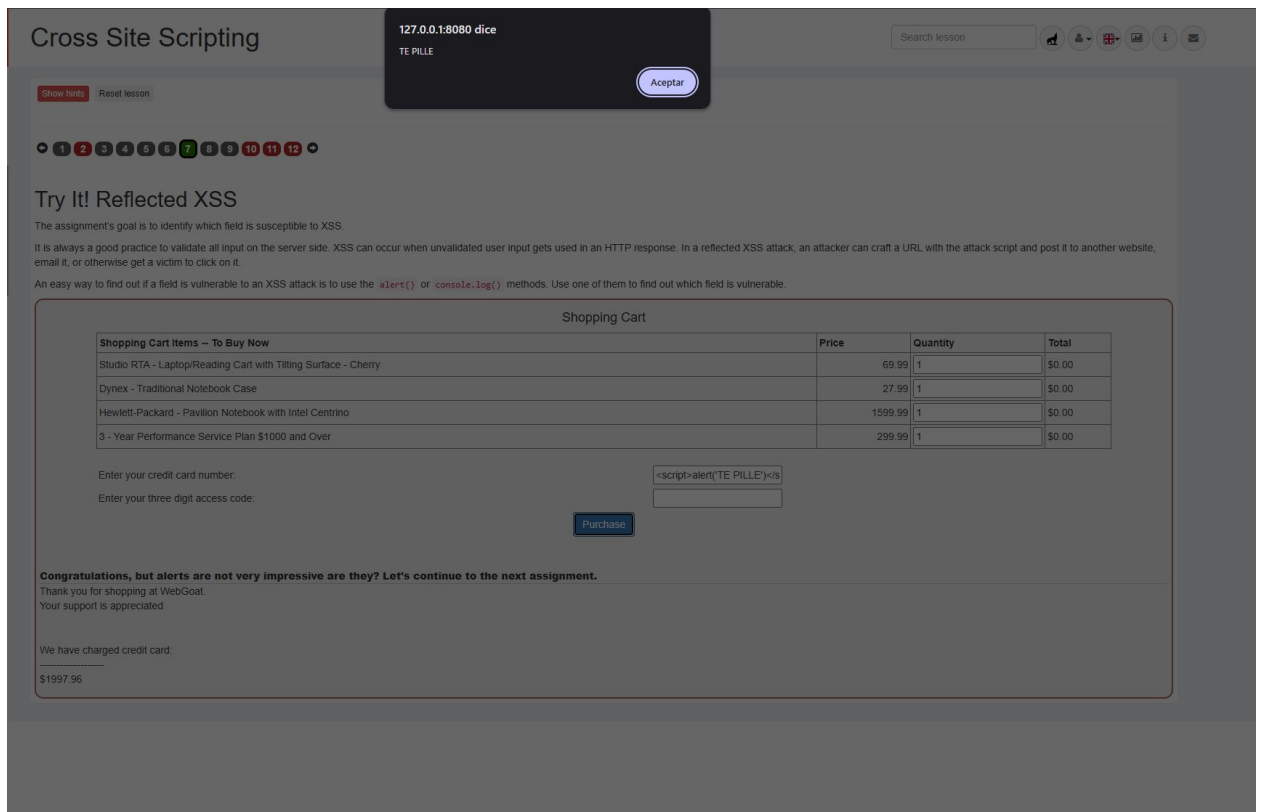
Well done! Now you are earning the most money. And at the same time you successfully compromised the integrity of data by changing your salary!

USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN
37648	John	Smith	Marketing	9000000	3SL99A
96134	Bob	Franco	Marketing	83700	LO9S2V
89762	Tobi	Barnett	Development	77000	TA9LL1
34477	Abraham	Holman	Development	50000	UU2ALK
32147	Paulina	Travers	Accounting	46000	P45JSI

- **Conclusiones:** El sistema permite la ejecución de múltiples consultas en una sola línea, lo que permite a un atacante modificar cualquier dato de la base de datos sin autorización.
- **Recomendaciones / Mitigación:** Desactivar el soporte para consultas múltiples en el driver de conexión y emplear parámetros tipados en las consultas SQL.

3. A3 Injection - Cross Site Scripting (Apartado 7)

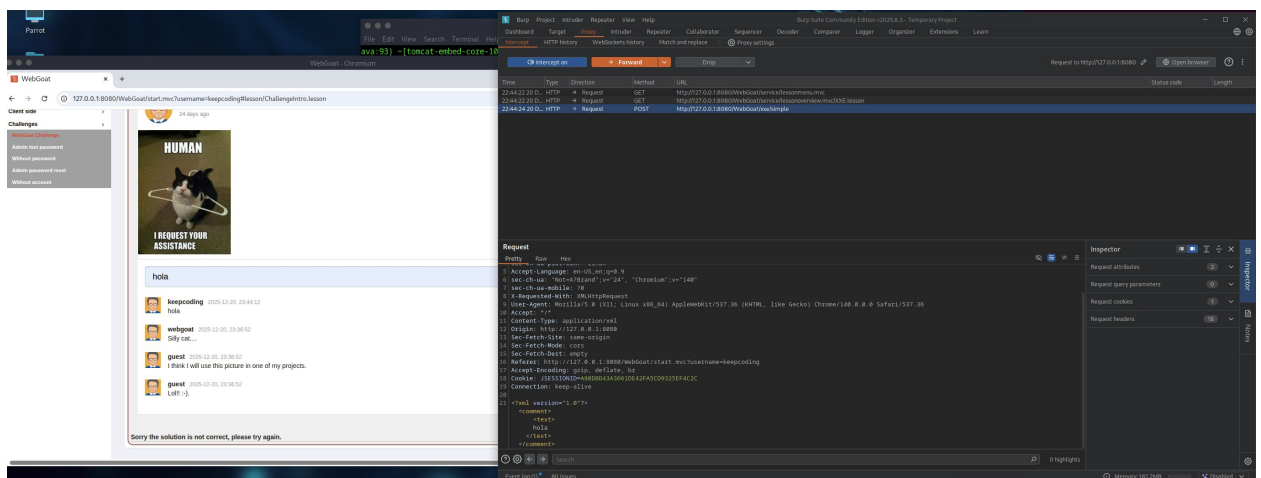
- **Proceso:** Se introdujo <script>alert('TE PILLE')</script> en el campo de la tarjeta de crédito. Al procesar el formulario, el navegador ejecutó el script de forma inmediata.

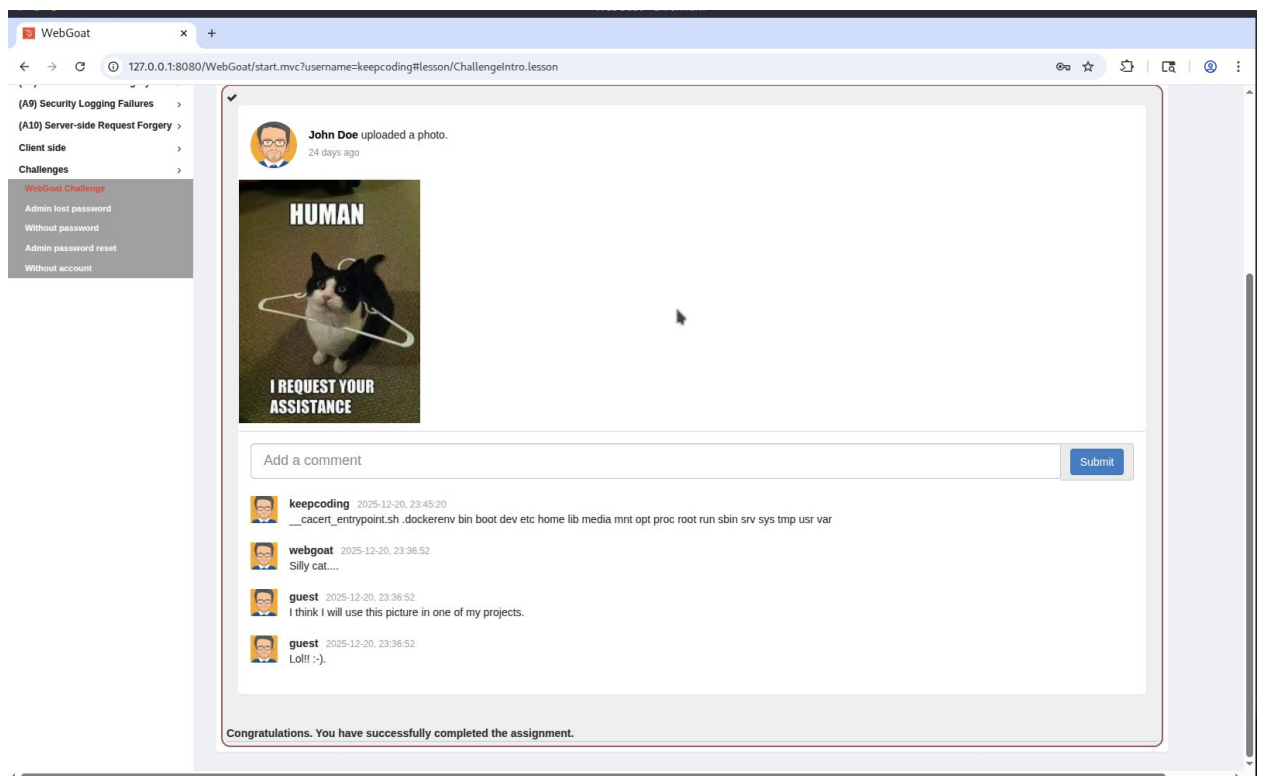
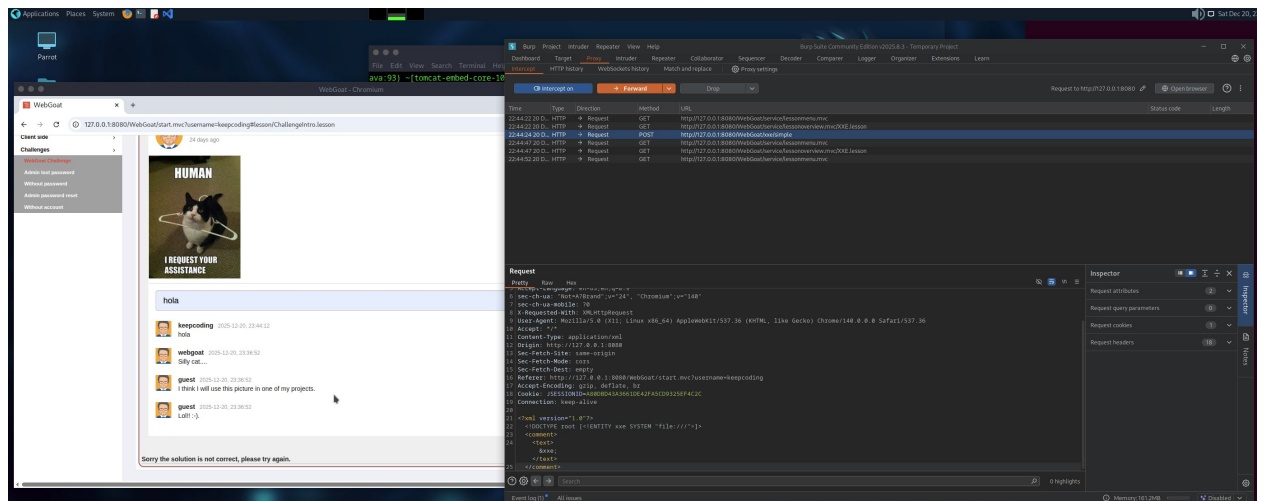


- **Conclusiones:** El fallo es un XSS reflejado; el servidor devuelve el script al navegador del usuario sin codificarlo, lo que podría usarse para robar cookies de sesión.
- **Recomendaciones / Mitigación:** Implementar **codificación de salida (Output Encoding)** para que caracteres como < y > se conviertan en entidades HTML seguras.

4. A5 Security Misconfiguration - XXE (Apartado 4)

- **Proceso:** Mediante **Burp Suite**, se modificó una petición XML para definir una entidad externa que apunta a la raíz del sistema: `<!ENTITY xxe SYSTEM "file:///"/>`.





- **Conclusiones:** El procesador de XML del servidor permite entidades externas, lo que resulta en una vulnerabilidad de **Local File Inclusion (LFI)** que expone archivos del sistema operativo.
- **Recomendaciones / Mitigación:** Configurar el procesador XML para **deshabilitar DTDs** y entidades externas completamente.

5. A6 Vuln & Outdated Components (Apartado 5)

- **Proceso:** Se identificó el uso de la librería **jquery-ui:1.10.4**, comparando su comportamiento con la versión **1.12.0**.

WEBGOAT Vulnerable Components

Search lesson

Reset lesson

1 2 3 4 5 6 7 8 9 10 11 12 13

The exploit is not always in "your" code

Below is an example of using the same WebGoat source code, but different versions of the jquery-ui component. One is exploitable; one is not.

jquery-ui:1.10.4

This example allows the user to specify the content of the "closeText" for the jquery-ui dialog. This is an unlikely development scenario, however the jquery-ui dialog (TBD - show exploit link) does not defend against XSS in the button text of the close dialog.

Clicking go will execute a jquery-ui close dialog: `OK<script>alert('XSS')</script>` Go!

This dialog should have exploited a known flaw in jquery-ui:1.10.4 and allowed a XSS attack to occur

jquery-ui:1.12.0 Not Vulnerable

Using the same WebGoat source code but upgrading the jquery-ui library to a non-vulnerable version eliminates the exploit.

Clicking go will execute a jquery-ui close dialog: `OK<script>alert('XSS')</script>` Go!

This dialog should have prevented the above exploit using the EXACT same code in WebGoat but using a later version of jquery-ui.

- **Conclusiones:** El uso de componentes antiguos hereda vulnerabilidades ya documentadas (CVEs) que pueden ser explotadas por terceros.
- **Recomendaciones / Mitigación:** Mantener un inventario de dependencias y establecer una política de actualizaciones periódicas a versiones estables y seguras.

6. A7 Identity & Auth Failure - Secure Passwords (Apartado 4)

- **Proceso:** Se realizó un bypass del sistema de recuperación de cuenta del usuario "admin" utilizando la respuesta predecible "green" para su color favorito.

WEBGOAT Password reset

Search lesson

Reset lesson

1 2 3 4 5 6 7

Security questions

This has been an issue and still is for a lot of websites, when you lost your password the website will ask you for a security question which you answered during the sign up process. Most of the time this list contains a fixed number of question and which sometimes even have a limited set of answers. In order to use this functionality a user should be able to select a question by itself and type in the answer as well. This way users will not share the question which makes it more difficult for an attacker.

One important thing to remember the answers to these security question(s) should be treated with the same level of security which is applied for storing a password in a database. If the database leaks an attacker should not be able to perform password reset based on the answer of the security question.

Users share so much information on social media these days it becomes difficult to use security questions for password resets, a good resource for security questions is: <http://goodsecurityquestions.com/>

Assignment

Users can retrieve their password if they can answer the secret question properly. There is no lock-out mechanism on this 'Forgot Password' page. Your username is 'webgoat' and your favorite color is 'red'. The goal is to retrieve the password of another user. Users you could try are: 'tom', 'admin' and 'tany'.

✓ Sign up Login

WebGoat Password Recovery

Your username

admin

What is your favorite color?

green

Submit

Congratulations. You have successfully completed the assignment.

- **Conclusiones:** Las preguntas de seguridad son mecanismos débiles porque se basan en información que no es secreta y puede ser adivinada o investigada .
- **Recomendaciones / Mitigación:** Eliminar las preguntas de seguridad y sustituirlas por **Doble Factor de Autenticación (2FA)** o enlaces de recuperación enviados a canales verificados (email/móvil) .

C.

Post-explotación

Tras las pruebas, se confirmó que el acceso obtenido permite la persistencia en la base de datos y la capacidad de pivotar hacia el sistema operativo a través del fallo XXE, lo que otorga un control casi total sobre la información almacenada.

d.

Herramientas utilizadas

- **Nmap:** Escaneo de puertos y reconocimiento de servicios.
- **Burp Suite (Proxy):** Interceptación y manipulación de tráfico XML/HTTP.
- **WebGoat 8.1.0:** Entorno de auditoría vulnerable.